SIGSPATIAL 2026

Story-Style Speaker Script

Lexicographic Multi-Preference Routing for Priority-Aware Urban Navigation

Target talk length: approximately 13–15 minutes

The story spine

Imagine a driver in Manhattan with one simple request:

"Safety first. If two routes are almost equally safe, then prefer the usual road. And if traffic changes, adapt."

Everything in the talk answers one part of that request: why current navigation cannot express it well, how urban data becomes preference scores, how those scores become a lexicographic route decision, why a near-tie tolerance is useful, and how Lexicographic FreqA* makes the result fast enough for interactive routing.

Full Speaker Script

Slide 1 - Title

35–45 sec

Good morning everyone. I'm Abdallah Abdelwahab from the University of Alabama, and today I'll present our work on **Lexicographic Multi-Preference Routing for Priority-Aware Urban Navigation**.

I want to start with one question:

Why should the navigation system decide what matters most to the user?

Imagine that I am driving in Manhattan at night. I may care about travel time, but maybe tonight safety matters more. And if two routes are almost equally safe, maybe I would rather take the road that drivers commonly use.

The problem is that current navigation systems do not really let me express that priority order.

That is the idea behind this work.

Transition: *So first, what is missing from the navigation systems we already use?*

Slide 2 - The Problem

45–55 sec

Most navigation services optimize mainly around travel time and expose a small number of binary switches: avoid tolls, avoid highways, avoid ferries.

Those controls are useful, but they do not capture many real preferences.

A parent driving late at night may want the safest route. A courier may care about congestion. Another driver may prefer fewer controlled intersections.

And the more important limitation is this: when these preferences conflict, the user cannot clearly say

which one should win.

For our Manhattan driver, saying "safety first" is fundamentally different from saying "give safety a weight of 0.6."

Transition: *That observation leads to the central idea of our approach.*

Slide 3 - Our Insight

40–50 sec

People naturally express preferences as an **order**.

They say:

"Safety first, then the usual way."

They usually do not think in a vector of carefully tuned weights.

So we let the user provide an ordered list of preferences.

At strict lexicographic priority, we examine the first preference first. A lower-priority preference cannot compensate for a clear loss on a higher-priority preference.

Later, I will show why we introduce a small tolerance so that lower preferences can still matter when two candidates are almost tied.

Transition: *Turning that simple user request into a routing algorithm creates two challenges.*

Slide 4 - Two Obstacles

50–60 sec

The first challenge is **representation**.

Safety, historical popularity, congestion, and intersection delay come from different sources, in different units, and they do not naturally live on the road graph in the same form.

So we need to convert all of them into a common scoring framework.

The second challenge is **optimization**.

A weighted sum can hide the user's priority because enough gain on a lower-priority criterion can compensate for a loss on the top one.

Pareto methods preserve alternatives, but they usually give the user a set of routes to choose from.

We want one answer:

one route, one stated priority order, interactive response time.

Transition: *Here is how the whole system fits together.*

Slide 5 - System Overview

55–65 sec

This diagram is the full pipeline.

On the left are the urban data sources: OpenStreetMap attributes, crash records, historical route information, and traffic information.

Offline, we align these sources with the road network, annotate the graph, and build the tile-level information used by our heuristic.

Online, the user gives us a source, a destination, and an ordered preference list.

If live traffic is available, it can update the traffic component at query time.

Then Lexicographic FreqA* searches the graph and returns a single priority-aware route.

So the pipeline has two jobs: first, translate the city into preference scores. Second, search those scores efficiently in the user's priority order.

Transition: *Let's look at the first part: what do these preference fields actually look like?*

Slide 6A - Preference Fields

35–45 sec

These three maps show the spatial fields we are working with.

Crash exposure is concentrated on particular corridors. Historical usage follows a different pattern. Signal density is widespread but still spatially nonuniform.

The key point is that these are **different signals over the same city**.

We transform each source into values in the range $(0, 1]$, where larger values are more desirable.

A route then accumulates these values along the path.

Transition: *But the route itself is small on this city-scale map, so let me zoom in on what the search is actually choosing.*

Slide 6B - Route Magnification

30–40 sec

Here the magnified regions show the source, destination, and the route segment more clearly.

Now the intuition becomes visible.

If safety is the primary preference, the score pushes the route away from high crash-exposure regions.

For historical usage, the route is encouraged toward commonly used segments.

For the traffic-related field, the same mechanism favors the less penalized corridor.

So the heatmap is not just visualization. It is the signal that shapes the path.

Transition: *To make that precise, we separate what belongs to a road segment from what belongs to an intersection.*

Slide 7 - Edge and Node Factors

40–50 sec

Each preference can have an **edge factor**, a **node factor**, or both.

Historical popularity is edge-based, so its node factor is simply one.

Safety has both components: mid-block crashes affect road segments, and intersection crashes affect nodes.

Traffic also has both: the road-speed or congestion signal is on the edge, while the intersection-control penalty is on the node.

This distinction is useful because the cost of entering an intersection can be different from the desirability of the road segment itself.

Transition: *Now we can look at how each of these factors is built from the original data.*

Slide 8 - Building the Factors

65–80 sec

I will not read every equation here. The important part is the intuition.

For **safety**, more recorded crashes reduce the score. Fatal crashes receive additional weight. Mid-block crashes are assigned to edges, while intersection crashes are assigned to nodes. The edge values are normalized over the outgoing choices.

For **historical popularity**, we ask: among historical routes heading toward the destination tile, how often was this edge used? We condition on a destination tile rather than an exact destination node so that the statistics are more stable.

For **traffic**, the live edge factor uses current speed divided by free-flow speed. So a segment moving near free-flow receives a high score, while congestion pushes the score down.

At intersections, signals, stops, yields, and crossings apply node-level penalties.

Most importantly, safety, traffic, and historical popularity remain **separate preference scores**. We do not multiply the three preferences together.

Transition: *So how do these local factors become a complete route decision?*

Slide 9 - From Factors to Route Selection

55–65 sec

There are three steps.

First, one traversal from u to v combines the edge factor with the node factor of the intersection we enter.

Second, we multiply those transition scores along the path. That gives us one complete route score for one preference.

Because the factors are in $(0, 1]$, a larger route score is better.

Third, for multiple preferences, we keep the scores in a vector: one component for safety, one for traffic, one for historical popularity, depending on the user's order.

Then we compare those components lexicographically.

For our running example, if the user says "safety first, then historical," safety is examined first.

Transition: *Strict lexicographic comparison is correct, but it creates one practical issue.*

Slide 10 - Why Introduce Epsilon?

55–65 sec

These route scores are real-valued products.

That means two candidates are almost never **exactly** equal on the first preference.

So with strict comparison, the primary preference may distinguish almost every candidate and the secondary preferences rarely get a chance to contribute.

We therefore introduce a relative tolerance, ε.

If two values on the current preference are within this relative tolerance, we treat them as a near-tie and move to the next preference.

If $\varepsilon = 0$, we recover strict lexicographic priority.

With a small positive ε, a lower preference can act only inside the near-tie region.

This gives us a controlled way to make the full preference order meaningful.

Transition: *Now we need to search this lexicographic objective without losing interactive performance.*

Slide 11 - Lexicographic FreqA* 65–75 sec

The method combines two ideas.

The first is **lexicographic search**. Instead of carrying one scalar score, each state carries a vector with one component per preference.

The second idea is the **tile-graph heuristic**.

We coarsen the road network into tiles and compute an optimistic estimate of the best remaining score toward the destination.

This plays the same role that an admissible heuristic plays in classical A*: it focuses the search on promising parts of the graph.

The preference-specific heuristic information can be reused when the user changes the ordering.

At $\varepsilon = 0$, the method is provably lexicographically optimal.

Transition: *The full algorithm is on the next slide, but there are only three lines of reasoning I want you to remember.*

Slide 12 - Algorithm 40–50 sec

I am not going to read the pseudocode line by line.

There are three things to notice.

First, $\vec{g}(v)$ stores the accumulated preference-score vector from the source to the current node.

Second, $\vec{h}(v)$ is the optimistic tile-based estimate of what remains, and together they form the search priority vector $\vec{f}(v)$.

Third, epsilon enters in the **relaxation decision**, shown in the algorithm at line 16.

That is where we ask whether the newly reached score vector is better than the one already stored at

the node under the epsilon-lexicographic comparison.

The priority queue itself stays strictly ordered.

So the tolerance affects whether a candidate path replaces another one; it does not make the queue ordering non-transitive.

Transition: *With the method defined, the first question is simple: does changing the priority actually change the route?*

Slide 13 - Result 1: The Order Matters

45–55 sec

Yes - and quite dramatically.

Here we use the same origin and destination and compare routes shaped by different preferences.

The three offline preference-first routes have pairwise edge overlap of at most **2.6 percent**.

So these are not tiny variations around one central shortest path. They are materially different routes.

That matters because it shows that the preference order is actually doing something spatially meaningful.

The user is not just adjusting a number behind the scenes; the selected corridor changes.

Transition: *But flexibility is only useful if we can still answer quickly.*

Slide 14 - Result 2: Fast and Optimal

60–70 sec

For the single-preference FreqA* case, the mean query time is **0.42 milliseconds** on the 256-by-256 grid.

Compared with FreqDijkstra on the same historical-frequency objective, this is a **97.8-times speedup** and about **80 percent fewer node expansions**.

At the same time, FreqA* matches the optimal FreqScore on all 197 reachable queries.

For the multi-preference Lex-FreqA* search, the mean time is still only **1.13 milliseconds**.

So the key result here is not just that the heuristic is fast. It is that preference-aware routing remains compatible with interactive search.

Speaker caution: Do not present 0.42 ms as the Lex-FreqA* latency. It is the single-preference FreqA* result; Lex-FreqA* is 1.13 ms.

Transition: *And the same framework can also react when the city itself changes.*

Slide 15 - Result 3: Live Traffic

45–55 sec

Here we run the same traffic-first query at two different times using the TomTom live traffic feed.

The congestion pattern changes, and the selected route changes with it.

Nothing about the core search algorithm needs to be redesigned. We update the traffic factors and run the same framework.

The routing search itself stays around **2.2 milliseconds**.

That number is search time; traffic retrieval and background refresh are separate.

So the method is not limited to static urban information.

Transition: *Of course, honoring a preference can require a detour. So how much does that preference cost?*

Slide 16 - Result 4: The Cost of Preference

65–75 sec

Preference-aware routes can be longer than the shortest path.

Historical-first is about 20 percent longer in this experiment. Safety-first and traffic-first can require substantially larger detours.

That is not a search failure. It is the measurable cost of asking for a different objective.

We therefore also evaluate an optional distance budget.

The user can limit how far the preference-aware route is allowed to deviate from the shortest path.

As the budget becomes tighter, the detour drops, but we also lose some preference quality.

For example, at a budget multiplier of 2.0, the safety detour drops to about **40.5 percent**, while retaining about **85.3 percent** of the unconstrained safety score.

So the system can expose an interpretable tradeoff rather than hiding it.

Transition: *Before I close, I want to be clear about what these experiments do and do not establish.*

Slide 17 - Scope and Limitations

45–55 sec

There are three main limitations.

First, the historical route corpus is synthetic. So the popularity field is useful for evaluating the routing mechanism, but it should not be interpreted as a validated model of real driver behavior.

Second, our safety signal measures recorded crash exposure, not per-vehicle crash risk, because the crash counts are not normalized by traffic volume.

Third, the evaluation is currently limited to Manhattan.

These limitations affect the interpretation of the data fields.

They do not change the basic search framework, and richer real-world preference fields can be substituted as long as they satisfy the scoring assumptions.

Transition: *So if there is one idea I want you to remember from this talk, it is this.*

Slide 18 - Takeaway

40–50 sec

A user should be able to say:

“Safest first, then fastest.”

And the navigation system should return one route that reflects that order, instead of hiding the tradeoff inside a weighted sum or asking the user to choose from a menu of Pareto alternatives.

Lexicographic FreqA* gives us a framework for doing that with separate preference scores, strict lexicographic optimality at epsilon zero, efficient tile-based search, and support for live traffic.

The broader message is simple:

We want the user - not the routing system - to decide what matters first.

Thank you. I'm happy to take questions.

Three Sentences to Memorize

Opening
"The main question behind this work is simple: why should the navigation system decide what matters most to the user?"

Core contribution
"Instead of collapsing preferences into weights, we keep the preference scores separate and compare them according to the user's priority order."

Closing
"We want the user - not the routing system - to decide what matters first."

Delivery Notes

- Do not read equations symbol by symbol. Explain what each equation means.
- Do not read the pseudocode. Point to g, h, and the epsilon-relaxation line.
- Slow down on Slides 9–12; this is the conceptual center of the talk.
- Move faster through Slide 8 if time is short.
- If you need to cut 60–90 seconds, skip the pseudocode slide verbally and go directly from the method to Result 1.
- Be precise with latency: **0.42 ms = FreqA***, **1.13 ms = Lex-FreqA***, **2.2 ms = live-traffic routing search**.
- When discussing epsilon, say **near-tie tolerance** rather than suggesting strict lexicographic optimization is incorrect.